

Nuovo Mondo

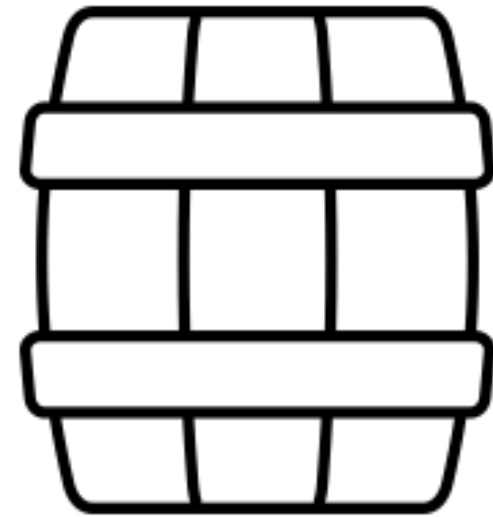
Documentazione tecnica

Damian Alex – Bettinelli Francesco – Brattico Thomas



Indice

- Introduzione al progetto;
- Librerie usate;
- Funzioni utilizzate;
- Strutture dati;
- Algoritmi principali;
- File di salvataggio & Non ripetizione eventi;
- Uso dell'IA;
- Test e riscontri;



Introduzione al progetto:

Nuovo Mondo è un gioco di simulazione. Il giocatore interpreta un avventuriero dotato di una nave e 2.000 monete, con l'obiettivo di raggiungere il Nuovo Mondo, barattare merci con gli indigeni e rientrare in patria massimizzando il profitto.

Il flusso di gioco si articola in sei fasi sequenziali: ingaggio della flotta, acquisto delle provviste, acquisto delle merci di baratto, viaggio (8 settimane stimate), baratto nel Nuovo Mondo e calcolo finale dei profitti. Durante la traversata possono verificarsi eventi casuali che alterano l'andamento della partita, introducendo un elemento di rischio non prevedibile.

Librerie usate:

Nel nostro programma le funzioni che ho usato sono:

- Random: per la scelta casuale degli eventi.
- Json: per salvare e caricare la partita.

Funzioni utilizzate:

- `mostra_menu_principale()` — stampa il menu iniziale con le opzioni "nuova partita" e "carica partita"
- `acquista equipaggio()` — gestisce l'ingaggio dei membri dell'equipaggio (almeno 1 per ruolo, max 16 totali), restituisce la lista dei membri
- `acquista_provviste(denari)` — permette di comprare verdura, frutta, carne e acqua scalando i denari, restituisce denari aggiornati e quantità acquistate
- `acquista_merci(denari)` — permette di comprare le merci da barattare (medicinali, armi, sale, ecc.), restituisce denari aggiornati e quantità acquistate
- `indice_provvista(nome)` — restituisce l'indice di una provvista nella lista `CATEGORIE_PROVVISTE` dato il nome
- `indice_merce(nome)` — restituisce l'indice di una merce nella lista `CATEGORIE_MERCI` dato il nome
- `conta_membri_vivi(lista_membri)` — restituisce il numero totale di membri ancora in vita
- `conta_membri_per_ruolo(lista_membri, nome_ruolo)` — conta quanti membri hanno un certo ruolo
- `rimuovi_membro_casuale(lista_membri)` — rimuove e restituisce un membro a caso dalla lista
- `rimuovi_n_membri_non_medici(lista_membri, numero_da_rimuovere)` — rimuove N membri casuali escludendo i medici, restituisce il numero effettivo di rimossi
- `aggiungi_membro(lista_membri, nome_ruolo, morale_iniziale, e_pagato)` — aggiunge un nuovo membro alla lista
- `frazione_a_testo(frazione)` — converte un valore decimale (es. 0.5) nella stringa frazionaria corrispondente (es. "1/2")
- `scegli_evento(contatore_eventi)` — seleziona casualmente un evento tra quelli disponibili, tenendo conto dei pesi e dei limiti di ripetizione
- `gestisci_evento(...)` — esegue la logica dell'evento estratto (tempesta, pirateria, albatro, ecc.) e restituisce le variazioni da applicare allo stato di gioco
- `controlla_scorte(...)` — consuma le provviste settimanali, avvisa se scarseggiano e permette di dimezzarle o raddoppiarle, aggiorna il delta morale
- `aggiorna_morale(lista_membri, delta_morale, variazione_evento)` — applica le variazioni di morale a tutti i membri e rimuove quelli che arrivano a 0, restituisce il numero di morti
- `mostra_riepilogo(...)` — stampa lo stato di fine settimana: membri vivi, morale, provviste e merci

Funzioni utilizzate:

- `calcola_ammutinamento(...)` — calcola il punteggio di rischio ammutinamento sommando o sottraendo punti in base alle condizioni di gioco, restituisce punteggio e cause
- `gestisci_ammutinamento(...)` — usa `calcola_ammutinamento` e determina se l'ammutinamento scatta (≥ 100 punti), restituisce True/False
- `ricalcola_settimane_per_morale(lista_membri, settimane_totali)` — se più della metà dell'equipaggio ha morale ≤ 30 , aggiunge una settimana al viaggio
- `viaggio(...)` — ciclo principale del viaggio: itera settimana per settimana, chiama eventi, scorte, morale e ammutinamento; restituisce l'esito e lo stato aggiornato
- `arrivo_nuovo_mondo(lista_membri, quantita_merci)` — gestisce lo sbarco; se si spara agli indigeni è game over, altrimenti si procede
- `esegui_baratto(quantita_merci)` — permette di barattare sale, stoffa, coltelli e diamanti con perle, manufatti o spezie, restituisce le quantità ottenute
- `gestisci_tradimento(...)` — evento opzionale: un rivale offre perle per le armi; se si accetta si rischia di essere scoperti (esito dipende dall'albatro)
- `epilogo_ritorno(lista_membri, quantita_provviste, ...)` — rifornisce la nave per il viaggio di ritorno e calcola le settimane necessarie
- `calcola_profitti(...)` — calcola il saldo finale sommando il valore dei beni indigeni venduti, sottraendo le paghe dell'equipaggio
- `esegui_asta()` — permette di vendere la nave tramite un'asta con offerte casuali per coprire eventuali debiti
- `mostra_esito_finale(saldo_finale)` — stampa il messaggio di fine gioco in base al saldo (positivo, nullo o negativo)
- `salva_partita(...)` — serializza lo stato completo della partita in un file JSON (`salvataggio.json`)
- `carica_partita()` — legge e deserializza il file di salvataggio, restituisce lo stato o None se non trovato
- `nuova_partita()` — orchestra l'intera partita da zero: equipaggio → provviste → merci → viaggio → nuovo mondo → profitti
- `partita_da_salvataggio()` — ripristina una partita dal file di salvataggio e la porta a conclusione
- `main()` — punto di ingresso: mostra il menu e avvia `nuova_partita` o `partita_da_salvataggio` in base alla scelta

Test e Riscontri

- **Scenari testati manualmente:**
 - Input non numerico in campo numerico → messaggio errore, ciclo riparte
 - Acquisto equipaggio con valore 0 o negativo → bloccato dalla validazione del range
 - Acquisto che supera FLOTTA_MASSIMA (16) → bloccato dal calcolo dinamico del massimo
 - Acquisto con denari insufficienti → messaggio errore, denari non modificati
 - Input diverso da s/n nelle scelte binarie → ciclo while lo rifiuta e ripete
 - Denari propagati correttamente tra fase 2 e fase 3 → verificato (pass-by-value)
 - Salvataggio e ricaricamento → stato ripristinato correttamente dalla settimana successiva
 - Ammutinamento con punteggio ≥ 100 → GAME OVER con cause elencate
 - Epidemia senza medici ne' medicinali → tutti gli ammalati muoiono
- **Problema riscontrato:**
 - Gestione del morale: il calcolo del delta accumulato generava valori incoerenti.
 - Risolto grazie alla spiegazione del professore e a successive sessioni di debug.

Strutture Dati – Panoramica

- Il codice usa esclusivamente liste e dizionari. Nessuna classe.
- **Liste parallele: stesso indice i per accedere a dati e quantita**
 - CATEGORIE_PROVVISTE[i] → nome, prezzo, consumo della provvista i
 - quantita_provviste[i] → quantita attuale di quella stessa provvista
- **Ogni membro dell'equipaggio e' un dizionario con 3 campi:**
 - {"ruolo": "cuoco", "morale": 100, "pagato": True}
 - ruolo: usato per conteggi e calcoli (es. numero medici)
 - morale: aggiornato ogni settimana; a 0 il membro muore
 - pagato: True = ingaggiato, False = recuperato in mare (non pagato)
- **contatore_eventi: dizionario che traccia quante volte ogni evento e' accaduto**
 - Confrontato con evento['limite'] per escludere eventi esauriti dal pool

Strutture Dati – Equipaggio & Provviste

- **Categorie equipaggio ingaggiabile (CATEGORIE_RUOLI)**
- `[{"nome": "cuoco", "prezzo_settimana": 15}, {"nome": "medico", "prezzo_settimana": 25}, ...]`
- Lista di dizionari: accesso per indice, ogni voce autodescrittiva, facilmente estendibile
- **Equipaggio ingaggiato (lista_membri)**
- `[{"ruolo": "cuoco", "morale": 100, "pagato": True}, ...]`
- Lista di dizionari: morale aggiornato ogni settimana; morti rimossi dalla lista
- `pagato=False` per i naufraghi recuperati in mare → non ricevono paga finale
- **Categorie scorte acquistabili (CATEGORIE_PROVVISTE)**
- `[{"nome": "verdura", "unita": "kg", "prezzo": 0.5, "consumo": 0.5}, ...]`
- Il campo consumo (unita/persona/settimana) evita costanti sparse nel codice
- **Scorte acquistate (quantita_provviste)**
- `[12.5, 8.0, 5.0, 3.0]` # verdura, frutta, carne, acqua
- Lista parallela: `indice_provvista(nome)` restituisce l'i da usare in entrambe le liste

Strutture Dati – Merci & Stato di Gioco

- **Categorie merci acquistabili (CATEGORIE_MERCI)**
 - `[{"nome": "armi", "descrizione": "armi", "prezzo": 5.0, "unita": "pezzo"}, ...]`
 - nome = chiave logica usata in TABELLA_BARATTO; descrizione solo per la stampa
- **Merchi acquistate (quantita_merci)**
 - `[5.0, 2.0, 10.0, 8.0, 6.0, 3.0]` # medicinali, armi, sale, stoffa, coltelli, diamanti
 - Lista parallela a CATEGORIE_MERCI; azzerata dopo il baratto
- **Tabella baratto (TABELLA_BARATTO)**
 - `{"sale": {"perla": 0.5, "manufatto": 0.5, "spezia": 1.0}, "stoffa": {...}, ...}`
 - Dizionario annidato: accesso diretto {merce}{bene} senza ricerca sequenziale
 - Valore = quante unita di merce occorrono per 1 bene indigeno
- **Stato di gioco (variabili scalari + contatore)**
 - albatro_ucciso: bool → flag permanente, O(1), salvato in JSON
 - razioni_correnti: lista float → moltiplicatore razione per ogni provvista
 - delta_morale: int → variazione morale accumulata per settimana

Algoritmi Principali

- **1. Massimo acquistabile equipaggio**

- `massimo = FLOTTA_MASSIMA - acquistati - (ruoli_rimasti - 1)`
- Garantisce almeno 1 posto per ogni ruolo non ancora soddisfatto
- Es: max=16, acquistati=8, rimasti=2 → $massimo = 16 - 8 - 1 = 7$

- **2. Selezione evento casuale pesata**

- `random.choices(nomi_disponibili, weights=pesi_disponibili, k=1)`
- Peso 3 = 3x piu' probabile di peso 1. Eventi esauriti esclusi dal pool prima del sorteggio

- **3. Consumo provviste settimanale**

- `consumo = consumo_base * razioni_correnti[i] * numero_membri`
- Razioni dimezzabili fino a 4 volte (x0.5 ogni volta). Fabbisogno futuro = consumo * settimane_rimaste

- **4. Rischio ammutinamento (punteggio)**

- +30: razioni ridotte / nessun cuoco / albatro ucciso / nave sovraffollata (>12)
- +10 per ogni settimana extra oltre le 8 base | -20: albatro rispettato
- punteggio >= 100 → GAME OVER immediato

File di Salvataggio & Non Ripetizione Eventi

- **Formato: JSON (salvataggio.json)**
- Leggibile con qualsiasi editor | modulo json nativo Python | nessuna dipendenza esterna
- **Contenuto del file:**
 - `lista_membri, quantita_provviste, quantita_merci, settimane_totali`
 - `settimana_corrente, contatore_eventi, albatro_avvistati, albatro_ucciso`
 - `razioni_correnti, delta_morale, denari_rimasti`
- **Un solo file di salvataggio. Salvataggio proposto a fine ogni settimana tranne l'ultima.**
- **Non ripetizione degli eventi**
 - `contatore_eventi = {"Epidemia": 1, "Venti favorevoli": 1, ...}`
 - Ogni evento ha un campo limite nella LISTA_EVENTI (1 = unico, 3 = max 3 volte, 99 = illimitato)
 - `if contatore_eventi.get(nome, 0) < evento["limite"]:` # includi nel pool
 - `contatore_eventi` salvato nel JSON → non ripetizione garantita anche dopo ricaricamento
 - Eccezione: Avvistamento albatro (limite=3) | Nessun imprevisto (limite=99, sempre disponibile)

Uso dell'Intelligenza Artificiale

- **Strumento utilizzato: Claude, Google Gemini**
- **1. Debug**
 - Bug nel ciclo di validazione 'vuoi continuare ad acquistare': il programma procedeva ignorando input non validi. Causa: mancanza del while interno.
 - La soluzione e' stata compresa e integrata autonomamente nel codice.
- **2. Stesura della documentazione**
 - Supporto nella strutturazione e redazione della documentazione tecnica.
 - Tutte le informazioni derivano dall'analisi diretta del codice da parte del gruppo.
- In tutti i casi le soluzioni proposte dall'IA sono state analizzate, comprese dal gruppo di lavoro.

Test e Riscontri

- **Scenari testati manualmente:**
 - Input non numerico in campo numerico → messaggio errore, ciclo riparte
 - Acquisto equipaggio con valore 0 o negativo → bloccato dalla validazione del range
 - Acquisto che supera FLOTTA_MASSIMA (16) → bloccato dal calcolo dinamico del massimo
 - Acquisto con denari insufficienti → messaggio errore, denari non modificati
 - Input diverso da s/n nelle scelte binarie → ciclo while lo rifiuta e ripete
 - Denari propagati correttamente tra fase 2 e fase 3 → verificato (pass-by-value)
 - Salvataggio e ricaricamento → stato ripristinato correttamente dalla settimana successiva
 - Ammutinamento con punteggio ≥ 100 → GAME OVER con cause elencate
 - Epidemia senza medici ne' medicinali → tutti gli ammalati muoiono
- **Problema riscontrato:**
 - Gestione del morale: il calcolo del delta accumulato generava valori incoerenti.
 - Risolto grazie alla spiegazione del professore e a successive sessioni di debug.